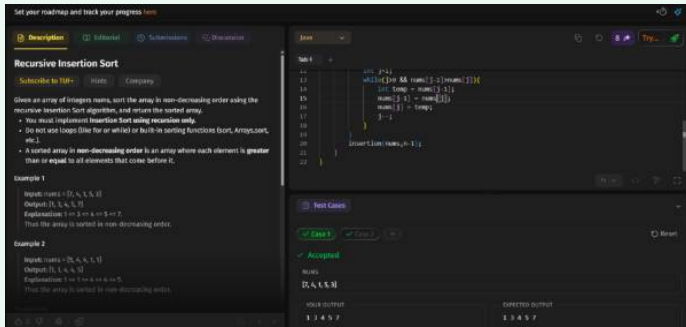


ChatGPT



Teach me Recursive Insertion sort

```

class Solution {
    public int[] insertionSort(int[] nums) {

        insertion(nums,nums.length);
        return nums;
    }
    private void insertion(int [] nums,int n){
        if(n<=1){
            return;
        }
        for(int i=0;i<=n-1;i++){
            int j=i;
            while(j>0 && nums[j-1]>nums[j]){
                int temp = nums[j-1];
                nums[j-1] = nums[j];
                nums[j] = temp;
                j--;
            }
        }
        insertion(nums,n-1);
    }
}

```

Show more Show less

Absolutely. The key idea is that **recursive insertion sort is just normal insertion sort where we replace one of the loops with recursion.**

Your current code is close, but it violates the problem requirement because it uses a **for** loop. It also does more work than necessary.

1. First understand normal insertion sort

Suppose:

[7, 4, 1, 5, 3]

Insertion sort thinks like this:

"I'll assume the first part is already sorted. Now I'll take the next element and insert it into its correct position."

So:

```
[7] 4 1 5 3
  ↑
sorted
Take 4 → insert before 7
[4, 7] 1 5 3
```

Then:

```
[4, 7] 1 5 3
Take 1 → move it left
[1, 4, 7] 5 3
```

Then:

```
[1, 4, 7] 5 3
Take 5 → insert between 4 and 7
[1, 4, 5, 7] 3
```

Finally:

```
[1, 3, 4, 5, 7]
```

2. Where does recursion come in?

The important observation is:

To sort n elements, first sort the first $n-1$ elements, then insert the n th element into the sorted portion.

That gives us the recursive structure:

```
insertionSort(n)
    sort first n-1 elements
    insert element n-1 into correct position
```

For:

```
[7, 4, 1, 5, 3]
```

the recursive calls conceptually become:

```

sort 5 elements
  sort 4 elements
    sort 3 elements
      sort 2 elements
        sort 1 element
          return
        insert element 1
      insert element 2
    insert element 3
  insert element 4

```

This is the heart of recursive insertion sort.

3. Your code's main problem

You have:

```

Java
for(int i=0;i<=n-1;i++){
  int j=i;
  while(j>0 && nums[j-1]>nums[j]){
    ...
  }
}
insertion(nums,n-1);

```

Notice the order:

```

do ALL the sorting with a loop
then recurse

```

That's not really using recursion to perform insertion sort.

You want:

```

recurse first
then perform ONE insertion

```

And the insertion itself should also avoid a loop if the problem says **recursion only**.

So we need two recursive ideas.

4. The two recursive functions

Think of it as:

Function 1: insertion(nums, n)

Responsible for sorting the first n elements.

```

Java
insertion(nums, n - 1);

```

Then:

```
insert nums[n-1]
```

Function 2: insert(nums, i)

Responsible for moving one element left until it reaches its correct position.

So:

```
insertion()
  ↓
sort smaller array
  ↓
insert last element
  ↓
insert() recursively moves it left
```

5. Recursive solution

```
Java
class Solution {
    public int[] insertionSort(int[] nums) {
        insertion(nums, nums.length);
        return nums;
    }

    private void insertion(int[] nums, int n) {
        // Base case
        if (n <= 1) {
            return;
        }

        // Sort first n-1 elements
        insertion(nums, n - 1);

        // Insert the nth element
        insert(nums, n - 1);
    }

    private void insert(int[] nums, int i) {
        // Base case:
        // element is already at the beginning
        if (i <= 0) {
            return;
        }

        // Correct position found
        if (nums[i - 1] <= nums[i]) {
            return;
        }

        // Swap with previous element
        int temp = nums[i];
        nums[i] = nums[i - 1];
        nums[i - 1] = temp;

        // Continue moving left
        insert(nums, i - 1);
    }
}
```

This uses **no for loop**, **no while loop**, and **no built-in sort**.

6. Let's trace it

Take:

```
[7, 4, 1, 5, 3]
```

We start:

```
Java
insertion(nums, 5);
```

It doesn't sort immediately.

It calls:

```
Java
insertion(nums, 4);
```

which calls:

```
Java
insertion(nums, 3);
```

which calls:

```
Java
insertion(nums, 2);
```

which calls:

```
Java
insertion(nums, 1);
```

Now:

```
Java
if (n <= 1)
    return;
```

So we hit the **base case**.

At this moment:

```
[7, 4, 1, 5, 3]
  ^
first 1 element is trivially sorted
```

Now recursion starts coming back

We return to:

```
Java
insertion(nums, 2);
```

First:

```
Java
insertion(nums, 1);
```

is already finished.

Now:

```
Java
insert(nums, 1);
```

We compare:

```
nums[0] = 7
nums[1] = 4
```

Since:

```
7 > 4
```

swap:

```
[4, 7, 1, 5, 3]
```

Then:

```
Java
insert(nums, 0);
```

Base case → return.

Next level

We're back at:

```
Java
insertion(nums, 3);
```

The first 2 elements are already sorted:

```
[4, 7, 1, 5, 3]
  ↑-----↑
  sorted
```

Now insert index 2:

```
1
```

Compare:

```
7 > 1
```

swap:

```
[4, 1, 7, 5, 3]
```

Then recursively:

```
Java
insert(nums, 1);
```

Compare:

```
4 > 1
```

swap:

```
[1, 4, 7, 5, 3]
```

Then:

```
Java
insert(nums, 0);
```

return.

Next level

Now:

```
[1, 4, 7, 5, 3]
```

First 3 elements are sorted.

Insert 5:

```
[1, 4, 7, 5, 3]
      ↑
```

7 > 5, swap:

```
[1, 4, 5, 7, 3]
```

Then compare 4 and 5:

```
4 <= 5
```

So stop.

Final level

Insert 3:

```
[1, 4, 5, 7, 3]
```

7 > 3:

```
[1, 4, 5, 3, 7]
```

5 > 3:

```
[1, 4, 3, 5, 7]
```

4 > 3:

```
[1, 3, 4, 5, 7]
```

1 <= 3, stop.

Done.

7. The most important mental model

Remember this:

```
insertion(n)
├── insertion(n-1)
│   └── sort smaller portion
└── insert(n-1)
    ├── compare with previous
    ├── swap if necessary
    └── recursively move left
```

Or even simpler:

insertion() makes the left side sorted. insert() puts the new element in its correct place.

That's the whole algorithm.

Base cases

There are two:

```
Java
if (n <= 1)
    return;
```

means:

An array with 0 or 1 element is already sorted.

And:

```
Java
if (i <= 0)
    return;

if (nums[i - 1] <= nums[i])
    return;
```

means:

Stop moving left when we've reached the beginning or the element is already in the correct position.

Complexity

For insertion sort:

- **Worst case:** $O(n^2)$
- **Best case:** $O(n)$
- **Space:** $O(n)$ due to recursion stack

The $O(n)$ space is important: iterative insertion sort normally uses $O(1)$ extra space, but recursive insertion sort creates recursive stack frames.

If you want to really master this, the next thing to understand is **why** `insertion(nums, n - 1)` **must happen before** `insert(nums, n - 1)`—that one detail makes the entire recursive solution click.

ChatGPT can make mistakes. Check important info.